

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Practical No: 4

Division: C Batch: 3

Roll No: 25 PRN No: 12410917

Name of Student: Shreyash S. Kamble

Subject Name: Deep Learning

Problem Statement:

Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm :

Theory

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network designed for sequential and time-series data. LSTM uses memory cells and gates to retain important information from previous time steps. This makes it suitable for forecasting problems where the current value depends on previous observations.

In this experiment, historical weather observations are provided to an LSTM model. The model uses the previous 24 hourly observations to predict the temperature for the next hour.

Dataset Description

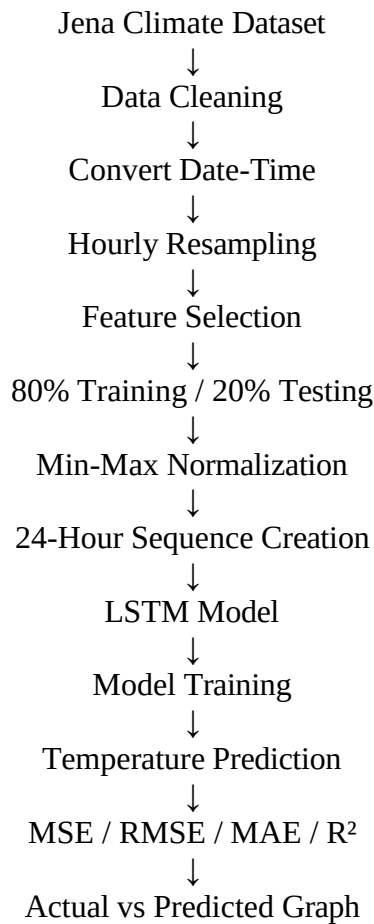
The Jena Climate dataset contains weather observations collected at the Max Planck Institute for Biogeochemistry weather station in Jena, Germany. The original data contains 14 weather variables recorded every 10 minutes over several years.

For this experiment, the data is resampled from 10-minute observations to hourly observations and the following variables are used:

Feature	Description
p (mbar)	Atmospheric pressure
T (degC)	Air temperature
Tpot (K)	Potential temperature
Tdew (degC)	Dew-point temperature
rh (%)	Relative humidity
wv (m/s)	Wind velocity
max. wv (m/s)	Maximum wind velocity

The target variable is T (degC).

Methodology



LSTM

LSTM stands for Long Short-Term Memory.

It is a type of Recurrent Neural Network designed to learn long-term dependencies in sequential data.

An LSTM cell consists mainly of:

1. Forget gate
2. Input gate
3. Cell state
4. Output gate

1. Forget Gate

The forget gate determines which information from the previous cell state should be discarded.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

where:

- f_t = forget gate
- x_t = current input
- h_{t-1} = previous hidden state
- σ = sigmoid activation

2. Input Gate

The input gate determines which new information should be stored.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

A candidate cell state is calculated as:

$$C_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

3. Cell State

The cell state is updated using the forget and input gates:

$$C_t = f_t * C_{t-1} + i_t * C_t$$

It acts as the memory of the LSTM.

4. Output Gate

The output gate determines what information is passed to the next layer.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

The hidden state is:

$$h_t = o_t * \tanh(C_t)$$

Thus, LSTM can selectively remember and forget information while processing sequential data.

Code :

```
import os
import zipfile
import urllib.request

import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# 1. DOWNLOAD DATASET
url = "https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip"

zip_path = "/content/jena_climate.zip"
extract_path = "/content/jena_climate"

if not os.path.exists(zip_path):
    urllib.request.urlretrieve(url, zip_path)

if not os.path.exists(extract_path):
    os.makedirs(extract_path)

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_path)

# 2. FIND CSV FILE
csv_file = None

for root, dirs, files in os.walk(extract_path):
    for file in files:
        if file.endswith(".csv"):
            csv_file = os.path.join(root, file)
            break

if csv_file is None:
    raise FileNotFoundError("CSV file not found.")

print("Dataset:", csv_file)

# 3. LOAD DATASET
df = pd.read_csv(csv_file)

print("\nDataset Shape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nFirst 5 Rows:")
print(df.head())

```

4. DATE-TIME PROCESSING

```
df["Date Time"] = pd.to_datetime(df["Date Time"])
```

```
df = df.set_index("Date Time")
```

```
df = df.sort_index()
```

5. SELECT FEATURES

```
features = [  
    "p (mbar)",  
    "T (degC)",  
    "Tpot (K)",  
    "Tdew (degC)",  
    "rh (%)",  
    "wv (m/s)",  
    "max. wv (m/s)"  
]
```

```
data = df[features].copy()
```

6. HANDLE MISSING VALUES

```
data = data.interpolate()
```

```
data = data.dropna()
```

7. RESAMPLE INTO HOURLY DATA

```
data = data.resample("1h").mean()
```

```
data = data.interpolate()
```

```
data = data.dropna()
```

```
print("\nHourly Dataset Shape:")
```

```
print(data.shape)
```

```
print("\nHourly Data:")
```

```
print(data.head())
```

8. VISUALIZE TEMPERATURE

```
plt.figure(figsize=(14, 5))
```

```
plt.plot(data.index, data["T (degC)"])
```

```
plt.title("Hourly Temperature")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Temperature (°C)")
```

```

plt.grid(True)
plt.show()

# 9. TRAIN-TEST SPLIT
split_index = int(len(data) * 0.80)

train_data = data.iloc[:split_index]
test_data = data.iloc[split_index:]

print("\nTraining Samples:", len(train_data))
print("Testing Samples:", len(test_data))

# 10. SCALE DATA
scaler = MinMaxScaler()

train_scaled = scaler.fit_transform(train_data)

test_scaled = scaler.transform(test_data)

# 11. CREATE SEQUENCES
lookback = 24

target_index = features.index("T (degC)")

def create_sequences(dataset, lookback):
    X = []
    y = []

    for i in range(lookback, len(dataset)):
        X.append(dataset[i-lookback:i])
        y.append(dataset[i, target_index])

    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_scaled, lookback)

X_test, y_test = create_sequences(test_scaled, lookback)

print("\nX_train Shape:", X_train.shape)
print("y_train Shape:", y_train.shape)

print("X_test Shape:", X_test.shape)
print("y_test Shape:", y_test.shape)
# 12. BUILD LSTM MODEL
model = Sequential()

```

```
model.add(
    LSTM(
        64,
        return_sequences=True,
        input_shape=(X_train.shape[1], X_train.shape[2])
    )
)
```

```
model.add(Dropout(0.2))
```

```
model.add(LSTM(32))
```

```
model.add(Dropout(0.2))
```

```
model.add(Dense(16, activation="relu"))
```

```
model.add(Dense(1))
```

```
# 13. COMPILE MODEL
```

```
model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)
```

```
print("\nModel Summary:")
model.summary()
```

```
# 14. EARLY STOPPING
```

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)
```

```
# 15. TRAIN MODEL
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=64,
    validation_split=0.1,
    callbacks=[early_stop],
    shuffle=False,
    verbose=1
)
```

```
# 16. PREDICTION
```

```

y_pred_scaled = model.predict(X_test)

# 17. CONVERT BACK TO ORIGINAL TEMPERATURE
dummy_actual = np.zeros(
    (len(y_test), len(features))
)

dummy_pred = np.zeros(
    (len(y_pred_scaled), len(features))
)

dummy_actual[:, target_index] = y_test

dummy_pred[:, target_index] = y_pred_scaled[:, 0]

y_test_actual = scaler.inverse_transform(dummy_actual)[:, target_index]

y_pred_actual = scaler.inverse_transform(dummy_pred)[:, target_index]

# 18. CALCULATE METRICS
mse = mean_squared_error(
    y_test_actual,
    y_pred_actual
)

rmse = np.sqrt(mse)

mae = mean_absolute_error(
    y_test_actual,
    y_pred_actual
)

r2 = r2_score(
    y_test_actual,
    y_pred_actual
)

print("MODEL EVALUATION")

print("MSE :", mse)
print("RMSE :", rmse)
print("MAE :", mae)
print("R2 :", r2)

# 19. PLOT ACTUAL VS PREDICTED

plt.figure(figsize=(15, 6))

```

```
plt.plot(
    y_test_actual[:500],
    label="Actual Temperature"
)

plt.plot(
    y_pred_actual[:500],
    label="Predicted Temperature"
)

plt.title("Actual vs Predicted Temperature")

plt.xlabel("Test Time Steps")

plt.ylabel("Temperature (°C)")

plt.legend()

plt.grid(True)

plt.show()

# 20. TRAINING AND VALIDATION LOSS
plt.figure(figsize=(10, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Training and Validation Loss")

plt.xlabel("Epoch")

plt.ylabel("MSE Loss")

plt.legend()

plt.grid(True)

plt.show()

# 21. PREDICT NEXT HOUR
```

```

last_sequence = test_scaled[-lookback:]

last_sequence = last_sequence.reshape(
    1,
    lookback,
    len(features)
)

next_temp_scaled = model.predict(last_sequence)

dummy_next = np.zeros(
    (1, len(features))
)

dummy_next[0, target_index] = next_temp_scaled[0, 0]

next_temperature = scaler.inverse_transform(
    dummy_next
)[0, target_index]

print("NEXT HOUR TEMPERATURE")

print(
    "Predicted Temperature:",
    round(float(next_temperature), 2),
    "°C"
)

```

Output :

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip

13568290/13568290 ————— **0s** 0us/step Dataset

Shape: (420551, 15)

First 5 Rows:

	Date Time	p (mbar)	T (degC)	Tpot (K)	Tdew (degC)	rh (%)	\
0	01.01.2009 00:10:00	996.52	-8.02	265.40	-8.90	93.3	
1	01.01.2009 00:20:00	996.57	-8.41	265.01	-9.28	93.4	
2	01.01.2009 00:30:00	996.53	-8.51	264.91	-9.31	93.9	
3	01.01.2009 00:40:00	996.51	-8.31	265.12	-9.07	94.2	

4	01.01.2009 00:50:00	996.51	-8.27	265.15	-9.04	94.1
---	---------------------	--------	-------	--------	-------	------

	VPmax (mbar)	VPact (mbar)	VPdef (mbar)	sh (g/kg)	H2OC (mmol/mol)	\
0	3.33	3.11	0.22	1.94	3.12	
1	3.23	3.02	0.21	1.89	3.03	
2	3.21	3.01	0.20	1.88	3.02	
3	3.26	3.07	0.19	1.92	3.08	
4	3.27	3.08	0.19	1.92	3.09	

	rho (g/m**3)	wv (m/s)	max. wv (m/s)	wd (deg)
0	1307.75	1.03	1.75	152.3
1	1309.80	0.72	1.50	136.1
2	1310.24	0.19	0.63	171.6
3	1309.19	0.34	0.50	198.0
4	1309.00	0.32	0.63	214.3

Columns:

['Date Time', 'p (mbar)', 'T (degC)', 'Tpot (K)', 'Tdew (degC)', 'rh (%)', 'VPmax (mbar)', 'VPact (mbar)', 'VPdef (mbar)', 'sh (g/kg)', 'H2OC (mmol/mol)', 'rho (g/m**3)', 'wv (m/s)', 'max. wv (m/s)', 'wd (deg)']

	Date Time	p (mbar)	T (degC)	Tpot (K)	Tdew (degC)	rh (%)	\
0	2009-01-01 00:10:00	996.52	-8.02	265.40	-8.90	93.3	
1	2009-01-01 00:20:00	996.57	-8.41	265.01	-9.28	93.4	
2	2009-01-01 00:30:00	996.53	-8.51	264.91	-9.31	93.9	
3	2009-01-01 00:40:00	996.51	-8.31	265.12	-9.07	94.2	
4	2009-01-01 00:50:00	996.51	-8.27	265.15	-9.04	94.1	

	VPmax (mbar)	VPact (mbar)	VPdef (mbar)	sh (g/kg)	H2OC (mmol/mol)	\
0	3.33	3.11	0.22	1.94	3.12	
1	3.23	3.02	0.21	1.89	3.03	
2	3.21	3.01	0.20	1.88	3.02	
3	3.26	3.07	0.19	1.92	3.08	
4	3.27	3.08	0.19	1.92	3.09	

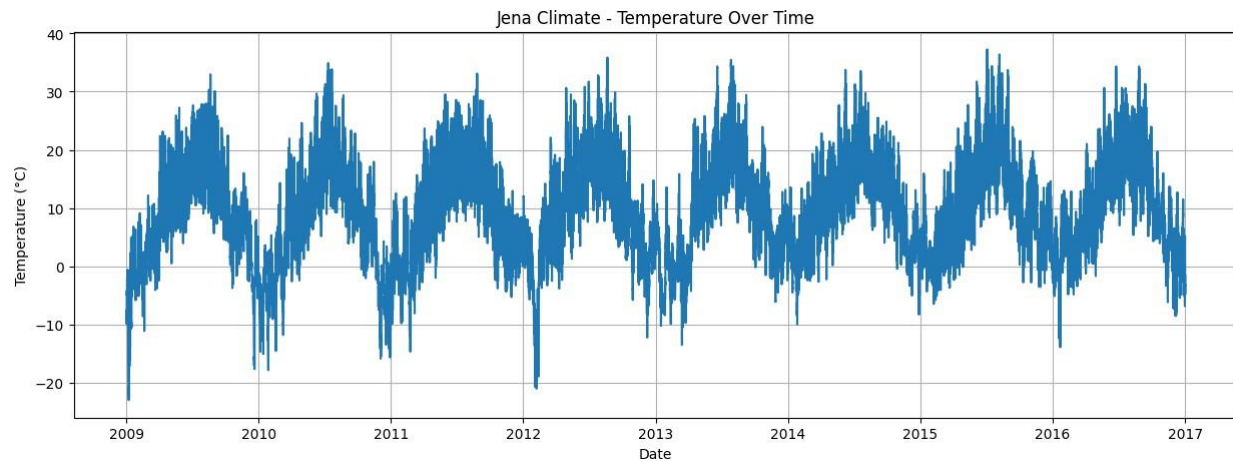
	rho (g/m**3)	wv (m/s)	max. wv (m/s)	wd (deg)
0	1307.75	1.03	1.75	152.3
1	1309.80	0.72	1.50	136.1
2	1310.24	0.19	0.63	171.6
3	1309.19	0.34	0.50	198.0
4	1309.00	0.32	0.63	214.3

	Date Time	p (mbar)	T (degC)	Tpot (K)	Tdew (degC)	rh (%)	\
--	-----------	----------	----------	----------	-------------	--------	---

420546	2016-12-31 23:20:00	1000.07	-4.05	269.10	-8.13	73.10
420547	2016-12-31 23:30:00	999.93	-3.35	269.81	-8.06	69.71
420548	2016-12-31 23:40:00	999.82	-3.16	270.01	-8.21	67.91
420549	2016-12-31 23:50:00	999.81	-4.23	268.94	-8.53	71.80
420550	2017-01-01 00:00:00	999.82	-4.82	268.36	-8.42	75.70

	VPmax (mbar)	VPact (mbar)	VPdef (mbar)	sh (g/kg)	H2OC (mmol/mol) \
420546	4.52	3.30	1.22	2.06	3.30
420547	4.77	3.32	1.44	2.07	3.32
420548	4.84	3.28	1.55	2.05	3.28
420549	4.46	3.20	1.26	1.99	3.20
420550	4.27	3.23	1.04	2.01	3.23

	rho (g/m**3)	wv (m/s)	max. wv (m/s)	wd (deg)
420546	1292.98	0.67	1.52	240.0
420547	1289.44	1.14	1.92	234.3
420548	1288.39	1.08	2.00	215.2
420549	1293.56	1.49	2.16	225.8
420550	1296.38	1.23	1.96	184.9



Hourly Dataset Shape: (70129, 14)

	p (mb ar)	T (de gC)	Tpot (K)	Tde w (de gC)	rh (%)	VP ma x (m bar)	VP act (m bar)	VP def (m bar)	sh (g/k g)	H2O C (mm ol/m ol)	rho (g/m* *3)	wv (m/ s)	ma x. wv (m/ s)	wd (deg)
Dat e Ti me														
200 9- 01- 01 00: 00: 00	996. 5280 00	- 8.3 040 00	265. 1180 00	- 9.12 000 0	93.7 800 00	3.2 600 00	3.0 580 00	0.2 020 00	1.9 100 00	3.06 8000	1309. 1960 00	0.5 200 00	1.0 020 00	174. 4600 00
200 9- 01- 01 00: 00: 00	996. 5250 00	- 8.0 650 00	265. 3616 67	- 8.86 166 7	93.9 333 33	3.3 233 33	3.1 216 67	0.2 016 67	1.9 516 67	3.13 3333	1307. 9816 67	0.3 166 67	0.7 116 67	172. 4166 67
200 9- 01- 01 02: 00: 00	996. 7450 00	- 8.7 633 33	264. 6450 00	- 9.61 000 0	93.5 333 33	3.1 450 00	2.9 400 00	0.2 016 67	1.8 366 67	2.95 0000	1311. 8166 67	0.2 483 33	0.6 066 67	196. 8166 67
200 9- 01- 01 03: 00: 00	996. 9866 67	- 8.8 966 67	264. 4916 67	- 9.78 666 7	93.2 000 00	3.1 116 67	2.8 983 33	0.2 100 00	1.8 116 67	2.90 6667	1312. 8133 33	0.1 766 67	0.6 066 67	157. 0833 33
200 9- 01- 01 04: 00: 00	997. 1583 33	- 9.3 483 33	264. 0266 67	- 10.3 450 00	92.3 833 33	3.0 016 67	2.7 750 00	0.2 316 67	1.7 333 33	2.78 0000	1315. 3550 00	0.2 900 00	0.6 700 00	150. 0933 33

Final Dataset Shape: (70041, 7)

Training Samples: 56032

Testing Samples: 14009

X_train: (56008, 24, 7)

y_train: (56008,)

X_test: (13985, 24, 7)

y_test: (13985,)

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 24, 64)	18,432
dropout (Dropout)	(None, 24, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

Total params: 31,393 (122.63 KB)

Trainable params: 31,393 (122.63 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

876/876 ————— **30s** 35ms/step - loss: 4.0734e-04 - mae: 0.0152 -

val_loss: 0.0050 - val_mae: 0.0670

Epoch 2/5

876/876 ————— **29s** 33ms/step - loss: 3.4958e-04 - mae: 0.0140 -

val_loss: 0.0040 - val_mae: 0.0594

Epoch 3/5

876/876 ————— **29s** 33ms/step - loss: 3.2214e-04 - mae: 0.0134 -

val_loss: 0.0034 - val_mae: 0.0539

Epoch 4/5

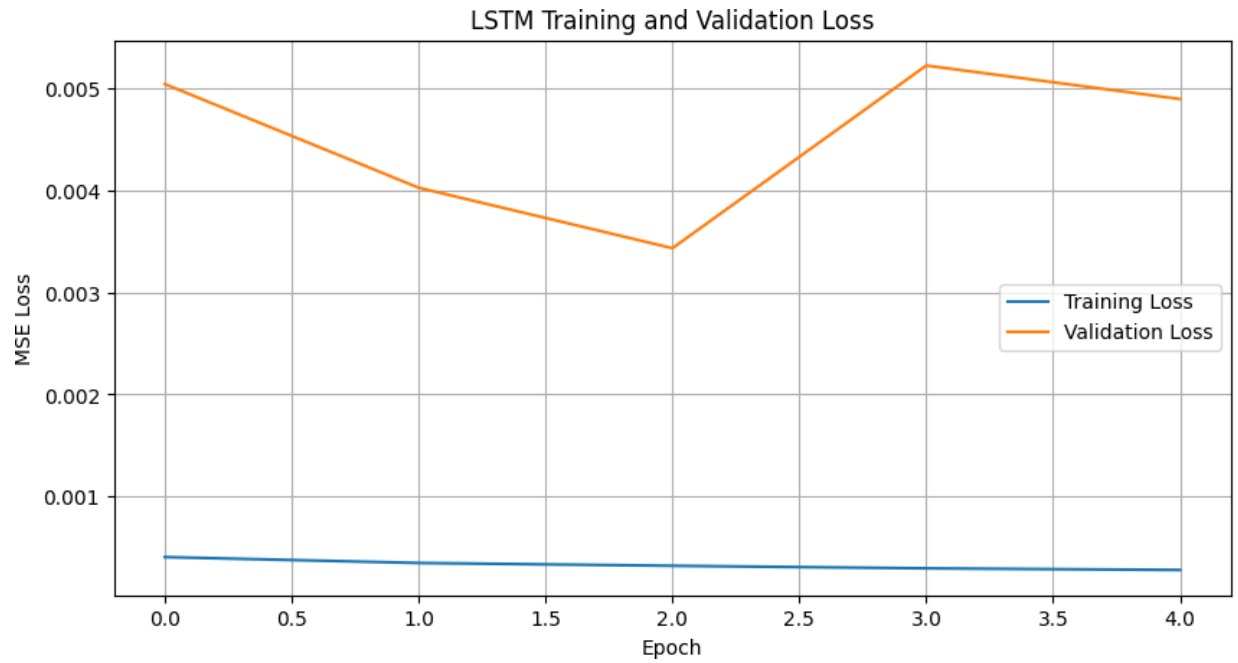
876/876 ————— **40s** 32ms/step - loss: 2.9700e-04 - mae: 0.0128 -

val_loss: 0.0052 - val_mae: 0.0654

Epoch 5/5

876/876 ————— **27s** 31ms/step - loss: 2.8117e-04 - mae: 0.0125 -

val_loss: 0.0049 - val_mae: 0.0624



438/438 ————— **3s** 7ms/step

===== LSTM

MODEL PERFORMANCE

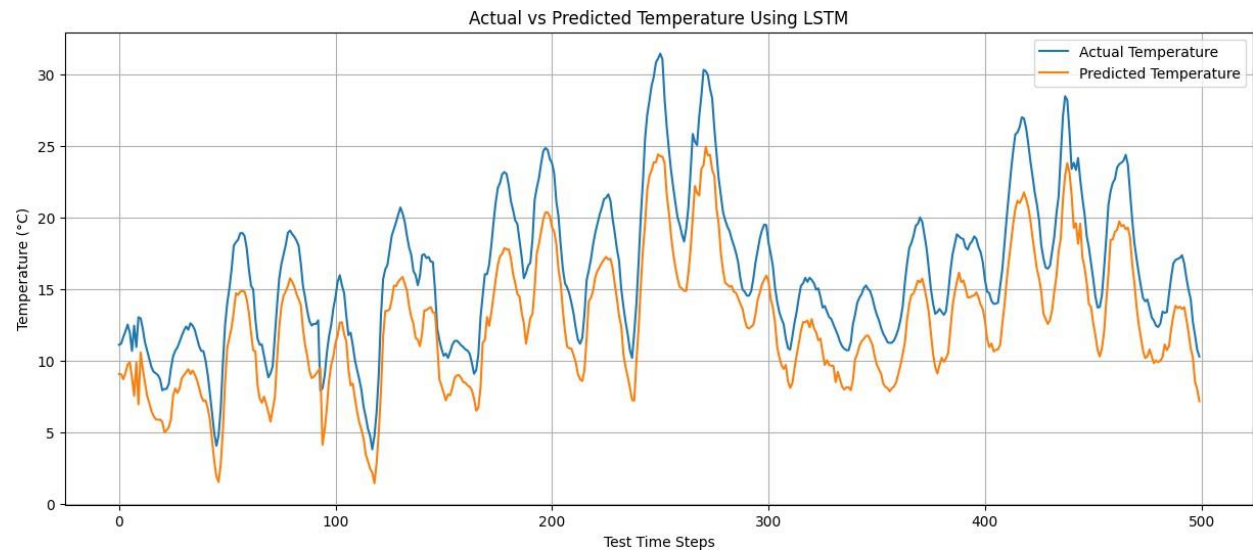
===== MSE :

11.6135

RMSE : 3.4079

MAE : 3.1376

R² : 0.8228



1/1 ————— 0s 41ms/step

Predicted Temperature for Next Hour: -6.71 °C